

Parcial 2 - 2024 - AdC

Ejercicio 1

Considere la ejecución del siguiente segmento de código LEGv8 para $i > 0$, donde $i \rightarrow X3$, y $X6$ contiene la dirección base del arreglo $A[]$ del tipo `uint32_t`.

El procesador tiene una CACHE exclusiva para **DATOS** de correspondencia **DIRECTA** de 64Kbyte y 4 palabras (w_p) por línea, sobre un procesador de 32bits, $CPI = 1$, que resuelve todos los data y control hazard sin necesidad de stalls, tiene una memoria principal de 4G palabras (w_m) de 1 byte cada una. Considerar que el resto de los registros están inicializados en 0 y la CACHE vacía al inicio de la ejecución del segmento.

```

LSL X9, X3, #2
ADD X9, X6, X9
loop: SUB X11, X6, X9
      CBZ X11, end
      LDUR X10, [X9, #0]
      ADD X0, x10, X0
      SUBI x9, x9, #8
      B loop
end:   STUR X0, [X9, #0]
    
```

a) Completar cada casillero con el número de bits de cada campo del formato de dirección de memoria principal:

Tag	Index	Word	Offset byte(*)
16	12	2	2

* Llenar con 0 en el caso que no corresponda

$$\text{Datos} = 64 \text{ KByte} = 2^6 \cdot 2^{10} \cdot 2^3 \text{ bits} = 2^{19} \text{ bits}$$

$$\text{Línea} = 4 w_p = 4 \times 32 = 2^2 \cdot 2^5 = 2^7$$

$$\# \text{ Líneas} = 2^{19} / 2^7 = 2^{12} \text{ 4K líneas}$$

$$\# \text{ Tags} = \frac{\# \text{ bloques de memoria}^*}{\# \text{ líneas de cache}} = \frac{2^{28}}{2^2 \cdot 2^{10}} = 2^{16} = 16 \text{ bits p/tag}$$

$$\# \text{ bloques de mem}^* = \frac{\# w_m}{\# w_m \text{ en un bloque}} = \frac{2^2 \cdot 2^{30}}{2^4} = 2^{28}$$

$4 \times \frac{32}{4B} = 16$

$$w_p = 4 w_m = 2 \text{ bits de offset}$$

$$\text{Línea} = 4 w_p = 2 \text{ bits de word}$$

$$46 = 2^{32} = \text{direcciones de 32 bits}$$

$$16 + 12 + 2 + 2 = 32$$

tag index w offset

b) Cuántos MISS de CACHE se produjeron en la ejecución del fragmento de código si $i = 6$ y la dirección base del arreglo A es $0x00F192B8$? Suponga que los accesos a memoria de escritura y lectura tienen **IDÉNTICO** comportamiento para la CACHE.

Respuesta: 3 Miss

Diagrama de código y anotaciones:

```

1 LSL X9, X3, #2
2 ADD X9, X6, X9
loop: 3 SUB X11, X6, X9
4 CBZ X11, end
5 LDUR X10, [X9, #0]
6 ADD X0, x10, X0
7 SUBI x9, x9, #8
8 B loop
end: 9 STUR X0, [X9, #0]
  
```

Anotaciones:

- 2^4 (arriba de la línea 1)
- $6 \times 2^2 = 24$ (arriba de la línea 2)
- $X6 + 24 = 0x00F192D0$ (punta a la línea 2)
- $0x00F192D0 - 8 = 0x00F192C8$ (punta a la línea 7)
- luego de ejecutar 3 veces esta línea tendré $X6 = X9$ (punta a la línea 7)
- Dirección base $+ 24 \Rightarrow$ acceso al elemento 6 (punta a la línea 2)
- (por $24/4 = 6$ me moví 6 veces 4 Bytes) (punta a la línea 2)

Accesos a mem:

Instru HIT/MISS Datos q traigo

5 it. 0 MISS $a[6], a[7], a[8], a[9]$

5 it. 1 MISS $a[2], a[3], a[4], a[5]$

5 it. 2 HIT (necesitaba $a[2]$)

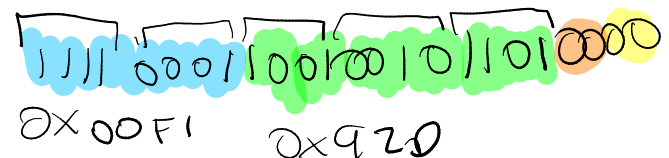
9 end MISS $?, ?, a[0], a[1]$

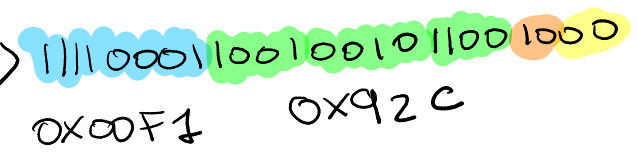
c) Completar como queda la memoria CACHE al finalizar la ejecución del segmento para las condiciones del punto b). Llenar una fila por cada MISS producido. Se completó la primer línea sólo con fines ilustrativos.

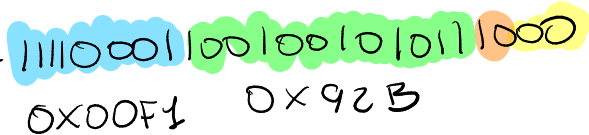
Index (hex)	Tag (hex)	V	Data			
			W ₃	W ₂	W ₁	W ₀
0xC0CA	0xCAFE	1	A[11]	A[10]	A[9]	A[8]

0x92D	0x00F1	1	A[9]	A[8]	A[7]	A[6]
0x92C	0x00F1	1	A[5]	A[4]	A[3]	A[2]
0x92B	0x00F1	1	A[1]	A[0]	—	—

Base = 0x00F192B8

1° acceso a memoria que da miss
 $0x00F192D0 \Rightarrow$ 

2° acceso a memoria que da miss
 $0x00F192C8 \Rightarrow$ 

3° acceso que da miss
 $0x00F192B8 \Rightarrow$ 

Ejercicio 2

En la figura se muestra un diagrama simplificado del procesador LEGv8 2-issue. Considerando la forma de funcionamiento de la implementación completa (con multiplexores, HDU, unidad de forwarding, los caminos de forwarding incluso a la etapa ID para la resolución de saltos, etc.) y *forwarding*, los caminos de forwarding incluso a la etapa ID para la resolución de saltos, etc.) y asumiendo que los saltos son perfectamente predichos (en el ciclo de clock posterior al *fetch* de un salto se hace *fetch* del paquete correcto: PC+8 o PC+offset, según corresponda, no hay penalidad), responder:

- a) Dado el siguiente fragmento de código en assembler LEGv8 (más abajo), analizar las dependencias de datos y completar con el **primer** caso que encuentre de cada una de las siguientes dependencias:

Tipo de dependencia	Números de instrucciones	Operando en conflicto
RAW	3 - 7	X2 ●
RAW condicional	1 - 6	X4 ●
WAW	5 - 8	X3 ●

- b) Mostrar en la tabla cómo organizaría los *issue packets* para ejecutar el programa en la menor cantidad posible de ciclos de clock **sin alterar el orden de las instrucciones** (cada instrucción sólo puede agruparse con la inmediata anterior, la inmediata posterior o una nop). El compilador asume toda la responsabilidad de insertar instrucciones nop para que el código se ejecute sin necesidad de generación de *stalls*. Usar los números correspondientes para referirse a las instrucciones del código y "nop" para las instrucciones agregadas.

Issue negro	Issue gris
-------------	------------

```

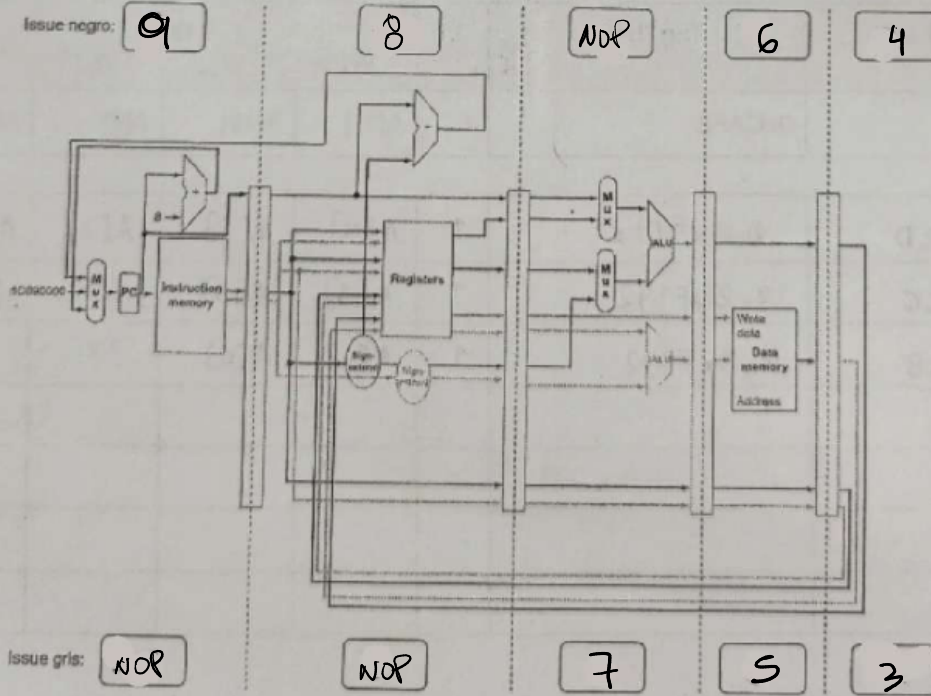
1> ORRI X4, XZR, #0x8
2> loop: CBZ X0, end
3> LDUR X2, [X1, #0]
4> SUBI X0, X0, #1
5> LDUR X3, [X1, #8]
6> ADD X1, X1, X4
7> STUR X2, [X1, #0]
8> ADD X3, X3, X1
9> CBZ XZR, loop
10> end:
    
```

NO Pasa
fetchear nada
porq la predicción
se hace el clk
luego del fetch
del salto

Issue negro	Issue gris
1	NOP
2	NOP
4	3
6	5
NOP	7
8	NOP
9	

Empiezo de clock 0

Completar en los recuadros vacíos del diagrama del procesador qué instrucción se está ejecutando en cada etapa del pipeline, tanto en el issue negro como en el gris, en el ciclo de clock número 6, considerando que antes de comenzar la ejecución del programa $X0=2$.



Issue negro	Issue gris
1	NOP
2	NOP
4	3
6	5
NOP	7
8	NOP
9	

	IF	ID	EX	MEM	WB
0	1				
1	2	1			
2	4,3	2	1		
3	6,5	4,3	2	1	
4	7	6,5	4,3	2	1
5	8	7	6,5	4,3	2
6	9	8	7	6,5	4,3

Ejercicio 4

Considerar que un procesador que cuenta con un predictor global de 8 bits ejecuta el siguiente código en C y compilado como se muestra a la derecha:

```
A[3]={3,7,4};
```

```
for (i = 0; i < 3; i++) { //X0 ← i
```

```
    if (A[i] < 5) { //X1 ← A[i]; X2 ← 5;
```

```
    ...
```

```
}
```

```
}
```

Dirección

```
0x14 L: ...
```

```
... ...
```

```
... cmp X1, X2
```

```
0x34 b.lt e_if
```

```
... ...
```

```
0x44 cbnz X0, L
```

fallen si $A[i] < 5$

Completar la siguiente tabla con todas las posiciones de la PHT que se modifican, considerar que todas las posiciones de memoria están inicializadas en 0 y el $GR=0x34$

GR / Dirección	Contenido
0011 0100	01
0110 1001	01
1101 0011	00
1010 0110	01
0100 1101	01
1001 1011	00

$A[0] < 5$

$i = 1, 1001$

$A[1] < 5$

$i = 2$

$A[2] < 5$

$i = 3$

Ejercicio 3

Considerando un microprocesador out-of-order execution implementado mediante el algoritmo de Tomasulo que cuenta con las características del hardware descritas en la tabla y que ejecuta el siguiente código en assembler, muestre el contenido de las tablas de estado de las Reservation stations y registros (en la próxima página) para el 4to clock de ejecución (inclusive y considerando que se hace fetch en el clk 0):

```

1> L1: SUB X0, X0, #8
2> LDUR X10, [X0, #0]
3> LDUR X11, [X1, #0]
4> ADD X1, X0, #8
5> ADD X12, X10, X12
6> MULT X12, X10, X12
7> STUR X12, [X0, #0]
8> STUR X12, [X1, #0]
9> CBNZ X0, L1
10> ORR X0, X1, X2
    
```

Hardware
El salto se predice correctamente
Issue = 4 instrucciones
Load = 4 RS / 1 clk
Store = 4 RS / 1 clk
ALU entera = 4 RS / 1 clk
ALU punto flotante = 4 RS / 2 clk
Multiplicación ^{ENTERA} punto flotante = 2 RS / 3 clk

Name	Reservation stations						
	Busy	Op	Vj	Vk	Qj	Qk	A
load 1	FALSE	ldur 2	[X0]	-	0	0	[X0 + #0]
load 2	FALSE	ldur 3	[X1]	-	0	0	[X1 + #0]
store 1	FALSE	stur 7	[X0]	-	0	0	[X1 + #0]
store 2	FALSE	stur 8	[X1]	-	0	0	[X0 + #0]
INT alu 2	FALSE	add 4	[X0]	#8	0	0	#0
INT alu 3	FALSE	add 5	-	[X12]	load 1	0	-
INT mul 1	FALSE	mult 6	-	-	load 1	INT alu 3	-
BRANCH	TRUE	CBNZ	[X0]	-8	0	0	-

Register Status											
	X0	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10
QI		INT alu 2									load 1

en etapa de write back, resultados disponibles al fin del ciclo